# AURASKIN

# AI SKIN INTELLIGENCE & PERSONALIZED SKINCARE PLATFORM

## PROJECT REPORT

**Submitted by**

**AKASH M D**

# TABLE OF CONTENTS


| Chapter No | Chapter Name | Page No |
|---|---|---|
| 1 | Introduction | 1 - 2 |
| 2 | System Architecture and Components | 3 - 5 |
| 3 | Methodology and Working of the System | 6 |
| 4 | Ingredient Intelligence and Product Recommendation | 7 - 13 |
| 5 | Testing, Deployment and Results | 12 |
| 6 | Conclusion and Future Enhancements | 15 - 17 |
| 7 | References | 18 |

# CHAPTER 1
# INTRODUCTION

## 1.1 Introduction

Skin health is influenced by several factors, including skin type, hydration, lifestyle, sleep, environmental exposure, skincare habits, and the suitability of cosmetic products. Selecting an appropriate skincare routine can be challenging because different individuals have different skin characteristics, concerns, and sensitivities. Traditional skincare practices often depend on general recommendations, trial-and-error product selection, and limited personal tracking. These approaches may not adequately consider the combined effects of lifestyle, environmental conditions, ingredient compatibility, and routine adherence.

AuraSkin, an AI Skin Intelligence and Personalized Skincare Platform, is developed to address these challenges through a centralized digital system that combines skin profile management, personalized recommendations, health scoring, ingredient intelligence, and progress tracking.

The platform evaluates user-provided information and generates an explainable skin health assessment. It provides personalized morning and evening skincare routines, ingredient information, product suitability scores, and progress reports.

The system also incorporates safety-oriented checks for reported allergies, sensitivity triggers, and potential ingredient conflicts. Its purpose is to support informed skincare decisions rather than replace professional medical diagnosis or treatment.

## 1.2 Problem Statement

Many skincare applications provide generic product suggestions without adequately integrating an individual's skin characteristics, lifestyle habits, hydration, sleep, environmental exposure, and existing skincare routine.

Users may also find it difficult to identify potentially unsuitable ingredients, understand product compatibility, maintain consistent skincare habits, and evaluate changes over time.

Therefore, there is a need for an integrated platform that evaluates multiple skin-related factors, explains its recommendations, considers reported sensitivities, and supports personalized skincare planning and progress monitoring.

AuraSkin aims to address these challenges by providing a unified platform that combines skin health assessment, ingredient analysis, personalized product recommendations, and progress tracking.

## 1.3 Objectives

The main objectives of the AuraSkin project are:

1. To develop a secure web-based platform for skin profile management and personalized skincare planning.
2. To implement a five-factor skin health scoring engine using user-provided skin and lifestyle information.
3. To generate personalized morning (AM) and evening (PM) skincare routines based on individual skin profiles and concerns.
4. To identify reported allergens, sensitivity triggers, and potential ingredient conflicts.
5. To develop an ingredient intelligence system that provides information about cosmetic ingredients and their suitability.
6. To implement a product recommendation engine that evaluates products according to skin type, concerns, safety, barrier support, and routine compatibility.
7. To generate explainable skin assessment reports with contributing factors and personalized recommendations.
8. To provide progress tracking and analytics that help users review changes in their recorded assessments and skincare habits.

9. To implement authentication and role-based access control for users, skincare consultants, dermatologists, and administrators.
10. To test the application modules, database integration, security controls, and deployment configuration.

## 1.4 Scope of the Project

AuraSkin focuses on the digital management and analysis of user-provided skin-related information. The platform includes user authentication, profile management, skin assessments, personalized skincare routines, ingredient education, product recommendations, progress analytics, and report generation.

The system evaluates multiple factors, including baseline skin condition, lifestyle, sleep, skincare routine adherence, and hydration. These factors are combined using a weighted scoring mechanism to generate an overall assessment score.

The platform also includes an ingredient intelligence engine, an INCI formula scanner, and a chemical interaction matrix to help identify potentially unsuitable ingredients and support product evaluation.

The product recommendation engine covers categories such as face wash, moisturizer, sunscreen, serum, toner, treatment products, and face masks.

AuraSkin uses a full-stack web architecture consisting of a React-based frontend, a FastAPI backend, and a PostgreSQL database. The application incorporates authentication, role-based permissions, persistent data storage, assessment reporting, and progress tracking.

The scope of the project is limited to personalized skincare guidance and decision support based on available user information and implemented rules. The platform does not independently establish a medical diagnosis, and its recommendations should not replace consultation with a qualified dermatologist when medical attention is necessary.

# CHAPTER 2
# SYSTEM ARCHITECTURE AND COMPONENTS

## 2.1 System Architecture

AuraSkin follows a client-server architecture in which the frontend communicates with the backend through Application Programming Interfaces (APIs). The backend processes requests, validates input data, executes assessment and recommendation logic, and interacts with the relational database.
The architecture is organized into several layers to improve modularity, maintainability, security, and scalability.

### 2.1.1 Presentation Layer

The presentation layer provides the user interface through which users interact with the application. It includes registration, login, profile management, assessment forms, dashboards, product recommendations, skincare routines, and reports.
The frontend is developed using React 18, TypeScript, and Vite. The interface follows a modern design system with responsive layouts and interactive components.

### 2.1.2 Application Layer

The application layer is implemented using Python and FastAPI. It receives frontend requests, validates input data, enforces authentication and authorization rules, and coordinates communication between the frontend, business logic, and database.
This layer provides endpoints for profile management, assessment processing, ingredient intelligence, product recommendations, and report-related operations.

### 2.1.3 Intelligence Layer

The intelligence layer contains the primary assessment and recommendation components. It includes the five-factor skin health scoring engine, risk assessment logic, ingredient intelligence engine, chemical interaction matrix, and product suitability scoring mechanism.
These components process available user information and generate structured assessment outputs and recommendations.

### 2.1.4 Data Layer

The data layer uses PostgreSQL for relational data storage and SQLAlchemy ORM for database operations.
It supports the persistence and retrieval of user information, skin assessments, product records, ingredient data, recommendations, and progress-related information.

### 2.1.5 Deployment and Monitoring Layer

The deployment layer includes application configuration, production server setup, database connectivity, logging, health checks, and deployment-related infrastructure.
Docker and Docker Compose configurations were included in the production-readiness work to support consistent application packaging and deployment.

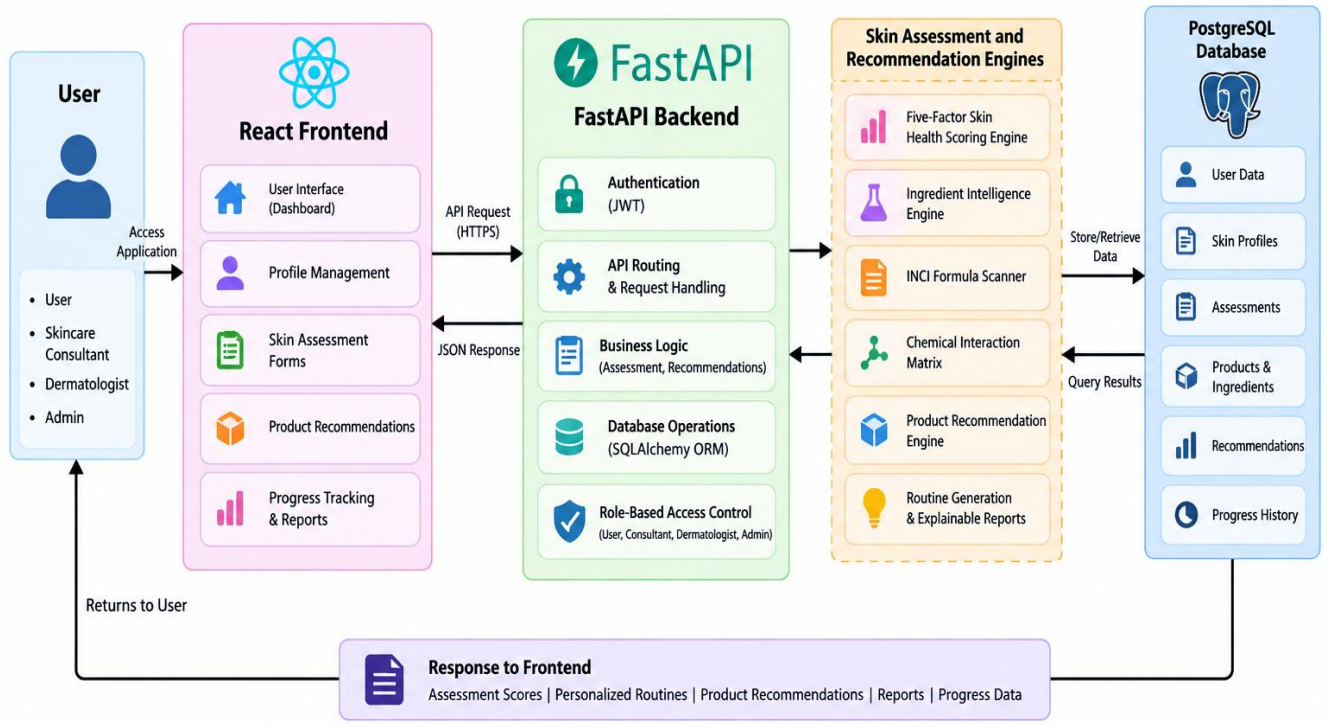



**Figure 2.1: System Architecture of AuraSkin**

## 2.2 Frontend Technologies

The frontend is developed using React 18, TypeScript, and Vite.
React provides a component-based approach to developing interactive user interfaces. It allows the application to be divided into reusable components such as forms, dashboards, assessment cards, product displays, and reports.
TypeScript provides static type checking, helping identify certain programming errors during development. Vite supports the frontend development environment and production build process.
The major frontend components include:

1. User registration and login pages.
2. User profile management.
3. Skin assessment forms.
4. Personalized dashboard.
5. Skincare routine interface.
6. Ingredient information and formula scanner.
7. Product recommendation interface.
8. Progress tracking and analytics.
9. Assessment reports and role-specific portals.

The interface uses a glassmorphism-inspired design system, responsive layouts, interactive dashboard elements, and feedback notifications.
Production optimization and code splitting were also included to reduce the amount of JavaScript loaded initially.

## 2.3 Backend Technologies

The backend is implemented using Python and FastAPI.
FastAPI provides API routing, request validation, structured responses, and integration with application services. Pydantic schemas support data validation, while SQLAlchemy manages database interactions through object-relational mapping.
The backend is responsible for:

- User authentication and authorization.
- Profile creation and management.
- Skin assessment processing.
- Five-factor scoring calculations.
- Personalized routine generation.
- Ingredient analysis and compatibility checks.
- Product suitability calculations.
- Progress data management.
- Assessment report generation.
- Application logging and health checks.

Gunicorn with Uvicorn workers is used in the production server configuration.

## 2.4 Database Design

PostgreSQL is used as the relational database for AuraSkin. SQLAlchemy ORM manages communication between the backend application and the database.
The database is designed to maintain persistent records for relevant application entities, including user accounts, profiles, assessments, products, ingredients, recommendations, and progress-related information.

The relational design supports data retrieval, updates, and relationships between application entities. Alembic migration architecture is included to manage database schema changes. This allows database structures to evolve in a controlled manner as application functionality changes.
Database indexing was also included in production optimization to improve the efficiency of selected database queries.

## 2.5 Authentication and Role-Based Access Control

AuraSkin implements JWT-based authentication and role-based access control.
Authentication verifies a user's identity, while authorization determines which resources and operations the authenticated user can access.
Passwords are handled using password hashing rather than plain-text storage. Protected endpoints require the appropriate authentication and authorization checks.
The system defines four primary roles.
1. USER
The user can manage personal profile information, complete assessments, view skincare recommendations, access routines, and review progress information.
2. SKINCARE_CONSULTANT
The skincare consultant role supports authorized consultation-related functionality according to the configured permissions.
3. DERMATOLOGIST
The dermatologist role supports authorized professional review functionality according to the implemented access controls.
4. ADMIN
The administrator manages authorized administrative operations, platform governance, and relevant

configuration functions.
Role-based permissions and user data isolation help prevent unauthorized access to protected information.

## 2.6 Technologies Used

| Component | Technology |
|---|---|
| Frontend | React 18 |
| Programming language | TypeScript |
| Frontend build tool | Vite |
| Backend | Python, FastAPI |
| Data validation | Pydantic |
| Database | PostgreSQL |
| ORM | SQLAlchemy |
| Database migrations | Alembic |
| Authentication | JWT-based authentication |
| Production server | Gunicorn and Uvicorn |
| Containerization | Docker and Docker Compose |
| Hosting | Render |

# CHAPTER 3
# METHODOLOGY AND WORKING OF THE SYSTEM

## 3.1 System Methodology

AuraSkin follows a structured workflow that converts user-provided information into personalized skincare guidance.
The system begins with user registration and authentication. After accessing the application, the user provides relevant skin profile information, concerns, lifestyle details, and other assessment inputs.
The application validates the information and processes it through the skin assessment engine. The engine calculates a weighted score and evaluates the available information according to the configured rules.
The system then applies relevant safety checks, generates personalized skincare routines, evaluates product suitability, and presents the results through the dashboard and assessment reports.
The major stages are:

1. User registration and authentication.
2. Skin profile creation.
3. Collection and validation of assessment information.
4. Five-factor skin health scoring.
5. Risk and sensitivity assessment.
6. Personalized AM and PM routine generation.
7. Ingredient and product suitability analysis.
8. Assessment report generation.
9. Progress tracking and updated assessments.

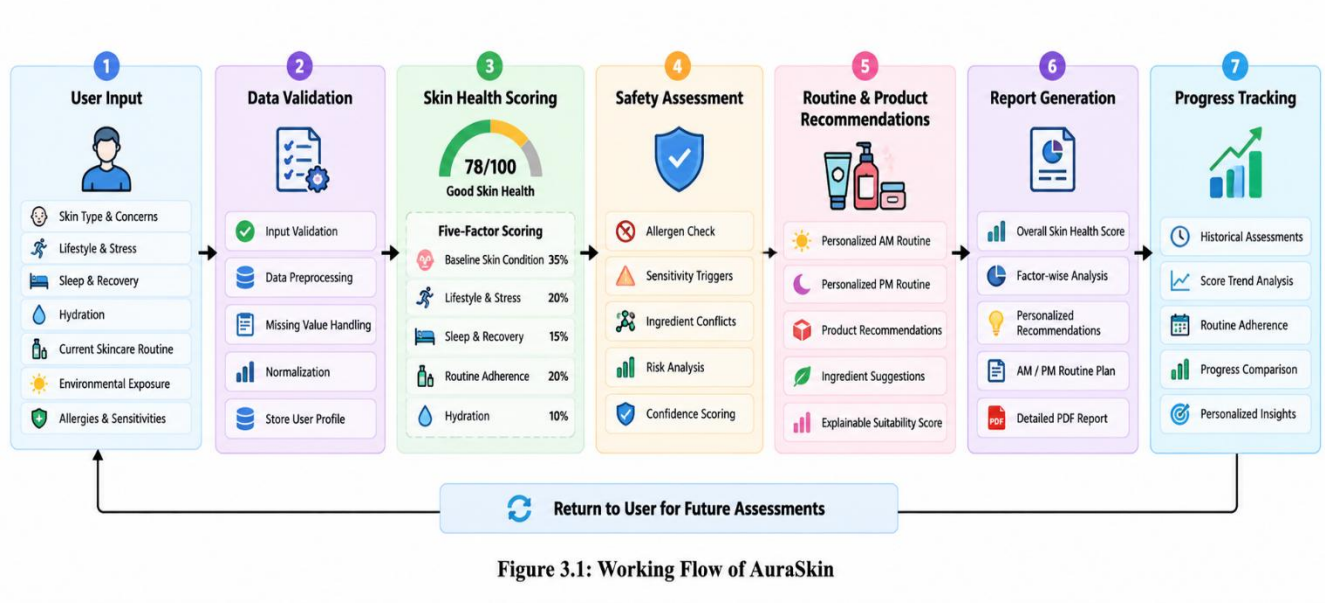



**Figure 3.1: Working Flow of AuraSkin**

## 3.2 User Profile and Data Collection

The user profile provides the information required for personalized assessment and recommendations. Depending on the fields available in the implemented application, the collected information includes skin type, reported concerns, lifestyle habits, sleep, hydration, existing skincare routine, environmental exposure, and reported allergies or sensitivity triggers.
The purpose of collecting this information is to establish a structured profile that can be used by the scoring and recommendation engines.

Input validation helps ensure that the application receives information in the expected format. Relevant data is stored in the database so that it can be retrieved for subsequent assessments and progress comparisons.
The completeness and accuracy of user-provided information influence the usefulness of the resulting recommendations.

## 3.3 Five-Factor Skin Health Scoring Engine

The five-factor skin health scoring engine is one of the core components of AuraSkin. It combines multiple dimensions of user-provided information into an overall score ranging from 0 to 100.
The initial scoring configuration uses the following factors and weights:

| Factor | Weight |
|---|---|
| **Baseline skin condition** | **35%** |
| **Lifestyle and stress** | **20%** |
| **Sleep and recovery** | **15%** |
| **Skincare routine adherence** | **20%** |
| **Hydration** | **10%** |
| **Total** | **100%** |

Let:

- (S) represent the normalized baseline skin condition score.
- (L) represent the lifestyle and stress score.
- (R) represent the sleep and recovery score.
- (A) represent the skincare routine adherence score.
- (H) represent the hydration score.

Each factor is represented on a normalized scale from 0 to 100.
The weighted assessment score is calculated as follows:

$$\begin{aligned}\text{Skin Health Score} ={}& 0.35S + 0.20L\\ &+ 0.15R + 0.20A + 0.10H\end{aligned}$$

The final score is bounded between 0 and 100 when all component scores are normalized to this range and the configured weights sum to one.
Each factor contributes to the final result according to its assigned weight. For example, the baseline skin condition contributes 35% of the total score, while hydration contributes 10%.
The resulting score provides a structured summary of the information processed by the application. It is a project-specific assessment score and should not be interpreted as a clinically validated measurement of skin health.

### 3.4 Risk Assessment and Confidence Scoring

After calculating the overall score, the assessment engine evaluates relevant concerns and contributing factors.
The system can identify areas that may require attention, such as reported dryness, sensitivity, inconsistent routine adherence, or incomplete hydration information.
The assessment output can include the overall score, contributing factors, risk-related indicators, and personalized suggestions.

Confidence indicators are intended to communicate the reliability of the assessment according to the implemented logic and available information. Confidence should not be interpreted as proof of diagnostic accuracy.
The assessment process is designed to make the results more understandable by presenting relevant contributing factors instead of relying only on a single numerical score.

## 3.5 Personalized Skincare Routine Generation

AuraSkin generates structured morning (AM) and evening (PM) skincare routines according to the user's profile, concerns, and the rules implemented in the recommendation engine.
A routine may include relevant steps such as cleansing, moisturizing, sun protection, and selected treatment products, depending on product suitability and the information available.
The routine generation process considers:

1. Skin type and reported concerns.
2. Product and ingredient suitability.
3. Reported allergies and sensitivity triggers.
4. Potential ingredient conflicts.
5. Routine compatibility.
6. The user's existing skincare practices.

The safety-oriented filtering process identifies relevant reported-allergen matches and potential compatibility concerns before recommendations are presented.
The resulting routines are intended to provide structured skincare guidance. They do not replace professional medical advice or individualized treatment prescribed by a qualified healthcare professional.

## 3.6 Explainable Assessment Reports

AuraSkin generates structured reports that help users understand their assessment results.
The reports can present the overall score, factor-level contributions, identified concerns, personalized recommendations, and suggested routine steps.
The reporting functionality supports the presentation of assessment information in a readable and organized format.
Explainability is important because users should be able to understand the factors contributing to a recommendation rather than receiving only a final score.
The reports are intended to support informed decision-making and progress review. Their usefulness depends on the quality of the input data and the correctness of the underlying scoring rules.
3.7 Progress Tracking
Progress tracking allows users to review recorded assessments and relevant skincare-related information over time.
The platform supports the presentation of historical information and progress-oriented indicators. This allows users to compare available assessment records and observe changes in their recorded scores or habits.
Historical data can help users review routine consistency and identify areas that may require further attention.
Progress tracking does not independently establish that a specific product or routine caused a change in skin condition. Such conclusions would require appropriate evaluation and evidence.

# CHAPTER 4
# INGREDIENT INTELLIGENCE AND PRODUCT RECOMMENDATION

## 4.1 Ingredient Intelligence Engine

The ingredient intelligence engine provides information about cosmetic ingredients and their relevance to skincare.
Cosmetic products may contain multiple ingredients with different functions. Understanding these ingredients can help users evaluate product formulations according to their skin profiles and reported sensitivities.
The AuraSkin ingredient intelligence engine connects available ingredient information with the product recommendation and safety-filtering components.
Its intended functions include:

1. Providing ingredient education.
2. Identifying ingredients in cosmetic formulations.
3. Flagging matches with reported allergens.
4. Identifying relevant sensitivity-related triggers.
5. Supporting potential ingredient compatibility checks.
6. Providing information for product suitability evaluation.

The reliability of the engine depends on the completeness, quality, and accuracy of the underlying ingredient knowledge base.

## 4.2 INCI Formula Scanner

The INCI Formula Scanner analyzes ingredient lists supplied in cosmetic product formulations.
INCI stands for International Nomenclature of Cosmetic Ingredients. It is a naming system used to identify cosmetic ingredients.
The scanner processes the available ingredient names and compares them with the platform's ingredient information.
The intended outputs include:

- Identification of recognized ingredients.
- Ingredient-related educational information.
- Potential matches with user-reported allergens.
- Sensitivity-related flags.
- Information that can support product suitability decisions.

The scanner helps users review product formulations before incorporating them into a routine.
A flagged ingredient does not necessarily indicate that every individual will experience an adverse reaction. Ingredient suitability can depend on individual sensitivities, concentration, formulation, and other factors.

## 4.3 Chemical Interaction Matrix

The Chemical Interaction Matrix organizes ingredient compatibility rules and potential conflicts.
Skincare formulations may contain multiple active ingredients, and some combinations may require caution depending on their properties and the individual's skin condition.
The matrix supports the identification of potential compatibility concerns according to the rules and ingredient information available in the system.
The recommendation engine can use these checks when evaluating products and generating skincare routines.
The matrix is intended as a decision-support component rather than a comprehensive clinical evaluation of every possible cosmetic interaction.

## 4.4 Product Recommendation Engine

The product recommendation engine evaluates products according to the user's profile, reported concerns, safety requirements, and routine compatibility.
The product catalog covers seven categories:

1. Face Wash
2. Moisturizer
3. Sunscreen
4. Serum
5. Toner
6. Treatment Products
7. Face Masks

The catalog includes product information and prices in Indian rupees.

### 4.4.1 Explainable Suitability Scoring

AuraSkin uses a suitability score ranging from 0 to 100% to support product evaluation.
The configured scoring factors are:

| Suitability factor | Weight |
|---|---|
| Skin type compatibility | 20% |
| Skin concern relevance | 25% |
| Allergen safety | 20% |
| Barrier support | 15% |
| Routine compatibility | 20% |
| Total | 100% |

Let:

- (T) represent skin type compatibility.
- (C) represent skin concern relevance.
- (A) represent allergen safety.
- (B) represent barrier support.
- (R) represent routine compatibility.

Each factor is normalized to a scale of 0 to 100.
The product suitability score is calculated as:

$$\begin{aligned}
\text{Suitability Score} ={}& 0.20T + 0.25C\\
&+ 0.20A + 0.15B + 0.20R
\end{aligned}$$

The weighted result supports comparisons between products according to the configured rules.
The suitability score is intended to explain how closely a product matches the available profile information. It does not establish clinical effectiveness.

### 4.4.2 Safety-Oriented Product Filtering

Product recommendations must consider reported allergies and sensitivity triggers alongside general suitability.
The platform incorporates safety-oriented filtering to identify relevant ingredient conflicts before presenting recommendations.
A high suitability score should not override an identified allergy or a relevant safety restriction.
Potential conflicts should be flagged according to the available information and implemented rules.

## 4.5 Product Categories and Personalization

The product recommendation system organizes available products into categories to make selection easier.
Face washes support cleansing, moisturizers support skin hydration, and sunscreens provide sun protection. Serums, toners, treatment products, and face masks serve different purposes depending on their formulations.
The platform evaluates products according to the information available in the catalog and the user's recorded profile.
The recommendations aim to help users compare products and identify options that align with their preferences and reported concerns.
The quality of the recommendations depends on accurate product descriptions, ingredient lists, suitability rules, and up-to-date catalog information.
4.6 Progress Analytics and Reporting
AuraSkin integrates assessment reporting and progress information with its recommendation workflow.
Users can review recorded assessment results and access relevant recommendations through the application interface.
Progress analytics can support comparisons between available assessment records and help users review changes in reported concerns or routine adherence.
Reports organize information into understandable sections so that users can review their scores, contributing factors, and suggested actions.
The platform provides informational support rather than a guarantee of a particular skincare outcome.

# CHAPTER 5
# TESTING, DEPLOYMENT AND RESULTS

## 5.1 Testing Methodology

Testing was performed during development and production optimization to evaluate the correctness of the backend, integration between application modules, and deployment readiness.
The testing process focused on verifying expected behaviour, identifying defects, and reducing the risk of regressions when new functionality was introduced.
The main testing areas included:

1. Authentication and authorization.
2. Input validation and API behaviour.
3. Skin assessment and scoring calculations.
4. Routine generation and safety filtering.
5. Ingredient intelligence and product recommendation.
6. Database persistence and retrieval.
7. Progress tracking and report generation.
8. Frontend and backend integration.
9. Production configuration and health checks.

The test suite provided a structured way to evaluate the application components against their expected behaviour.

### 5.2 Functional Testing

Functional testing verifies whether individual modules perform their intended operations.
In AuraSkin, the major functional areas include registration, login, profile management, skin assessment, personalized routines, ingredient scanning, product recommendations, and report generation.
Representative test cases are presented below.

| Test case | Expected behaviour |
|---|---|
| User registration | Valid registration data is processed correctly. |
| Authentication | Valid credentials permit authentication. |
| Input validation | Invalid data is rejected or handled according to validation rules. |
| Skin assessment | The engine calculates the configured weighted score. |
| Routine generation | Recommendations reflect available profile information and implemented rules. |
| Allergy filtering | Relevant reported-allergen conflicts are flagged. |
| Product suitability | The engine calculates the score using configured factors. |
| Database persistence | Valid records can be stored and retrieved. |
| Progress history | Previously saved assessment information can be accessed. |
| Access control | Protected resources enforce configured permissions. |

The expected results describe the intended behaviour of each test case. Actual test outcomes should be supported by the project's test execution logs.

## 5.3 Security Testing

Security testing focuses on authentication, authorization, password handling, and access to protected resources.
The application implements JWT-based authentication and role-based access control to restrict operations according to the configured user roles.
Security-related checks should verify that unauthorized users cannot access protected endpoints and that users cannot perform operations outside their assigned permissions.
Input validation and database access controls also contribute to application security.
The presence of security features does not independently establish that the application is free from vulnerabilities. Additional security reviews and testing are necessary for a complete assessment.

## 5.4 Integration Testing

Integration testing evaluates whether the frontend, backend, database, and intelligence modules communicate correctly.
The frontend sends requests to backend APIs, while the backend validates the requests, processes the relevant application logic, and retrieves or stores information in PostgreSQL.
The resulting responses are returned to the frontend for presentation.
Integration testing is important because individual modules may function correctly in isolation while still encountering problems when connected.
Relevant integration areas include authentication, profile persistence, assessment processing, recommendation retrieval, and report generation.

## 5.5 Backend Test Results

During the production optimization phase, the project development record reported the following backend test result:
Backend test suite: 83 out of 83 tests passing.
The recorded result consisted of 74 existing tests and 9 additional production-readiness and observability tests.
The result indicates that all tests in that recorded test suite passed at the time of execution.
It should not be interpreted as proof that every possible user scenario, security issue, external integration, or production failure has been tested.

## 5.6 Frontend Build Optimization

The frontend production build was optimized as part of the production-readiness work.
The recorded build results were:

| Metric | Recorded result |
|---|---|
| Production build errors | 0 |
| Initial bundle size | Approximately 525 kB |
| Optimized bundle size | Approximately 128 kB |
| Reported gzip size | 31.7 kB |

Code splitting and production build optimization were used to reduce the initial frontend bundle size.
A smaller bundle can reduce the amount of JavaScript that needs to be downloaded and processed, depending on the application structure and network conditions.
The build results document compilation and bundle-size improvements. They do not independently establish real-world page-load performance across all devices and network conditions.

## 5.7 Database and Production Optimization

Database and backend optimization were included in the final production-readiness work.
The project included:

- Database indexing.
- SQLAlchemy connection-pool configuration.
- Alembic migration architecture.
- Production server configuration using Gunicorn and Uvicorn workers.
- Application logging and health checks.
- Frontend production build optimization.
- Docker and Docker Compose configuration.
- Deployment-oriented environment configuration.

These measures support maintainability and deployment preparation.
Their effectiveness in a live environment depends on the final configuration, workload, database plan, resource limits, and operational monitoring.

## 5.8 Deployment Architecture

AuraSkin was prepared for cloud deployment using a frontend, backend, and relational database architecture.
The frontend is built with React, TypeScript, and Vite. The backend uses FastAPI and is served through Gunicorn with Uvicorn workers. PostgreSQL provides relational data persistence.
Render was selected for hosting the application. The deployment configuration included a PostgreSQL database service and a Python backend web service.
The backend configuration included environment variables, database connectivity, production server settings, and application logging.
Docker and Docker Compose were also included in the deployment-readiness work to support consistent packaging and configuration.

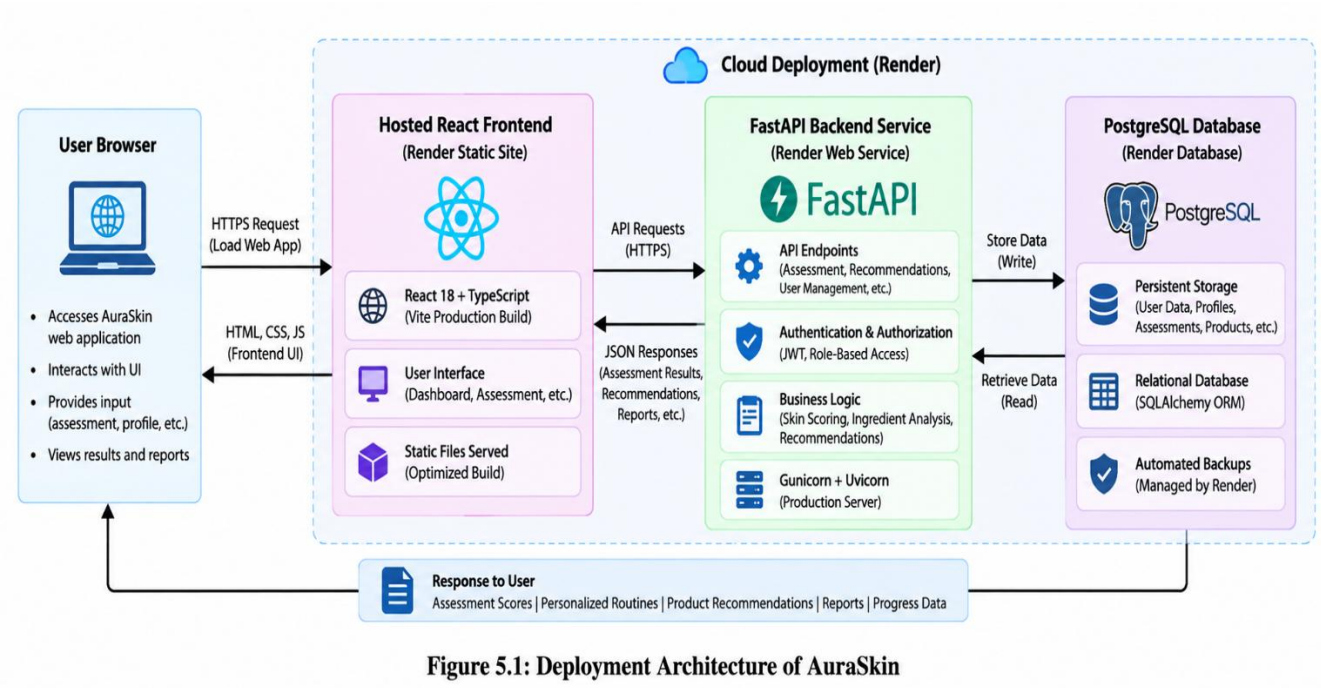



**Figure 5.1: Deployment Architecture of AuraSkin**

## 5.9 Deployment Challenges and Considerations

During deployment, the application requires correct configuration of the frontend API endpoint, backend service, database connection, authentication flow, and environment variables.
A successful frontend build does not necessarily mean that the complete application is functioning correctly in the hosted environment.
Similarly, a successful backend deployment does not guarantee that registration, login, assessment processing, and database operations all work as expected.
Therefore, end-to-end testing must be performed after deployment to verify the complete user journey.
The hosted application's operational status also depends on the hosting provider's service limits, database availability, resource allocation, and account configuration.

## 5.10 Results and Discussion

AuraSkin integrates several skincare-related functionalities into a single web application.
The main implementation outcomes include:

1. A full-stack web application using React, TypeScript, and FastAPI.
2. JWT-based authentication and role-based access control.
3. PostgreSQL persistence using SQLAlchemy ORM.
4. A five-factor skin health scoring engine.
5. Personalized AM and PM skincare routine generation.
6. Ingredient education and INCI formula scanning.
7. A chemical interaction matrix and safety-oriented filtering.
8. Product suitability scoring across seven product categories.
9. Assessment reports and progress tracking.
10. Production optimization, database indexing, and migration architecture.

The recorded development results include 83 passing backend tests and a frontend production build with no reported build errors.
The frontend bundle was reported to have been reduced from approximately 525 kB to 128 kB, with a gzip size of 31.7 kB.
These outcomes demonstrate the implemented software components and the recorded testing and optimization work. They do not independently establish clinical effectiveness, medical diagnostic accuracy, or universal production reliability.

# CHAPTER 6
# CONCLUSION AND FUTURE ENHANCEMENTS

## 6.1 Conclusion

AuraSkin, the AI Skin Intelligence and Personalized Skincare Platform, was developed to provide a structured and personalized approach to skincare planning.
The project integrates user profile management, multi-factor skin assessment, explainable scoring, personalized routine generation, ingredient intelligence, product suitability evaluation, progress tracking, and assessment reporting.
The five-factor assessment engine combines baseline skin condition, lifestyle and stress, sleep and recovery, skincare routine adherence, and hydration into a weighted score.
The product recommendation engine evaluates products according to skin type compatibility, concern relevance, allergen safety, barrier support, and routine compatibility. The ingredient intelligence components provide educational information and help identify relevant reported-allergen matches and potential compatibility concerns.
The application also incorporates authentication, role-based access control, relational database persistence, and production-oriented configuration.
Testing and optimization activities were carried out to evaluate backend functionality, integration, and deployment readiness. The recorded backend test suite achieved 83 passing tests during the production optimization phase. Frontend build optimization also reduced the reported bundle size.
Overall, AuraSkin demonstrates how a full-stack application can combine structured user data, configurable scoring rules, ingredient information, and explainable recommendations to support personalized skincare planning.
The quality of the platform's outputs depends on the accuracy of user-provided information, the reliability of ingredient data, and the correctness of the implemented scoring and recommendation rules.
AuraSkin is intended to support informed skincare decisions and does not replace professional medical diagnosis or treatment.

## 6.2 Future Enhancements

Although AuraSkin incorporates multiple assessment and recommendation features, several enhancements could extend the platform's capabilities.

### 6.2.1 Image-Based Skin Analysis

A future version could integrate appropriately trained and validated computer vision models to analyze user-submitted skin images.
Such functionality would require suitable datasets, privacy safeguards, evaluation across different skin tones, and careful validation before being used to provide assessment results.

### 6.2.2 Improved Ingredient Knowledge Base

The ingredient intelligence engine could be expanded using reliable scientific literature, validated cosmetic ingredient databases, and additional formulation information.
This could improve the coverage of ingredient identification and compatibility rules.

### 6.2.3 Enhanced Personalization

The recommendation engine could be improved through systematic evaluation, additional user feedback, and more detailed personalization rules.

Any changes to the scoring mechanism should be evaluated to ensure that recommendations remain consistent, explainable, and safety-conscious.

### 6.2.4 Advanced Progress Analytics

Future development could include more detailed longitudinal analysis of assessment history, routine adherence, and user-reported changes.
These features could help users review their progress over longer periods while avoiding unsupported claims about the causes of changes.

### 6.2.5 Expanded Testing and Validation

Additional testing could include usability studies, broader security assessments, load testing, and systematic evaluation of recommendation quality.
Clinical claims would require appropriate evidence and validation rather than relying solely on software testing results.

### 6.2.6 Deployment and Monitoring Improvements

The deployment infrastructure could be further improved through automated deployment pipelines, enhanced monitoring, backup procedures, and more comprehensive operational testing.
These improvements would support maintainability and help identify problems in the hosted environment.

# REFERENCES


1. FastAPI Documentation. *FastAPI Framework Documentation*. Available at: https://fastapi.tiangolo.com/
2. React Documentation. *React – The Library for Web and Native User Interfaces*. Available at: https://react.dev/
3. TypeScript Documentation. *TypeScript Handbook*. Available at: https://www.typescriptlang.org/docs/
4. PostgreSQL Documentation. *PostgreSQL Official Documentation*. Available at: https://www.postgresql.org/docs/
5. SQLAlchemy Documentation. *SQLAlchemy ORM Documentation*. Available at: https://docs.sqlalchemy.org/
6. Pydantic Documentation. *Data Validation Using Python Type Annotations*. Available at: https://docs.pydantic.dev/
7. Alembic Documentation. *Database Migration Tool for SQLAlchemy*. Available at: https://alembic.sqlalchemy.org/
8. Gunicorn Documentation. *Gunicorn Application Server Documentation*. Available at: https://docs.gunicorn.org/
9. Vite Documentation. *Vite – Next Generation Frontend Tooling*. Available at: https://vite.dev/
10. Render Documentation. *Deploying and Hosting Applications on Render*. Available at: https://render.com/docs
11. International Nomenclature of Cosmetic Ingredients (INCI). Ingredient naming and cosmetic formulation reference materials.
12. Relevant scientific publications and authoritative cosmetic ingredient references used in the implementation of the AuraSkin ingredient intelligence system should be added here.